

Data Structures and Algorithms – IT1170

Practical 4 – Queues / Answers

Exercise 1

1. Create a class QueueArray that implements a queue using an array and stores integers.
2. Implement the following methods:
 - insert(int item): Adds an item to the rear.
 - remove(): Removes and returns the front item.
 - peekFront(): Returns the front item without removing it.
 - isEmpty(): Checks if the queue is empty.
 - isFull(): Checks if the queue is full.
3. Introduce a new method named getCount() to get the number of elements in the Queue.
4. Write a main method to test the queue operations.

➤ Q1, Q2, Q3

```
class QueueArray {  
  
    private int[] queue;  
    private int front;  
    private int rear;  
    private int count;  
    private int maxSize;  
  
    public QueueArray(int size) {  
  
        maxSize = size;  
        queue = new int[maxSize];  
  
        front = 0;  
        rear = -1;  
        count = 0;  
    }  
  
    public boolean isEmpty() {  
        return count == 0;  
    }  
  
    public boolean isFull() {
```

```

        return count == maxSize;
    }

    public void insert(int item) {

        if(isFull()) {
            System.out.println("Queue is Full");
            return;
        }

        rear++;
        queue[rear] = item;
        count++;
    }

    public int remove() {

        if(isEmpty()) {
            System.out.println("Queue is Empty");
            return -1;
        }

        int temp = queue[front];
        front++;
        count--;

        return temp;
    }

    public int peekFront() {

        if(isEmpty()) {
            return -1;
        }

        return queue[front];
    }

    public int getCount() {
        return count;
    }
}

```

➤ Q4 Main Method

```
public class Main {  
  
    public static void main(String[] args) {  
  
        QueueArray q = new QueueArray(5);  
  
        q.insert(10);  
        q.insert(20);  
        q.insert(30);  
  
        System.out.println(  
            "Front = " + q.peekFront());  
  
        System.out.println(  
            "Removed = " + q.remove());  
  
        System.out.println(  
            "Number of Elements = "  
            + q.getCount());  
    }  
}
```

Exercise 2

5. Modify previously implemented methods to ensure the properties of circular queue.

- insert(int item): Adds an item to the rear and reassign rear to initial state after max index.
- remove(): Removes and returns the front item, when front reaches to max index reassign to initial index.
- peekFront(): Returns the front item without removing it, when front reaches to max index reassign to initial index
- isEmpty(): Checks if the queue is empty.
- isFull(): Checks if the queue is full.

6. Write a main method to test the queue operations. Allow the user to define the size of the circular queue. Initially, allow the user to insert values to all available indexes. Afterwards, remove values and insert values appropriately to showcase the behavior of circular queue

➤ Circular Queue Class

```

class CircularQueue {

    private int[] queue;
    private int front;
    private int rear;
    private int count;
    private int maxSize;

    public CircularQueue(int size) {

        maxSize = size;
        queue = new int[maxSize];

        front = 0;
        rear = -1;
        count = 0;
    }

    public boolean isEmpty() {
        return count == 0;
    }

    public boolean isFull() {
        return count == maxSize;
    }

    public void insert(int item) {

        if(isFull()) {
            System.out.println("Queue is Full");
            return;
        }

        rear = (rear + 1) % maxSize;

        queue[rear] = item;

        count++;
    }

    public int remove() {

        if(isEmpty()) {
            System.out.println("Queue is Empty");

```

```

        return -1;
    }

    int temp = queue[front];

    front = (front + 1) % maxSize;

    count--;

    return temp;
}

public int peekFront() {

    if(isEmpty()) {
        return -1;
    }

    return queue[front];
}
}

```

➤ Main Program

```

import java.util.Scanner;

public class CircularQueueTest {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.print(

            "Enter Queue Size: ");

        int size = input.nextInt();

        CircularQueue q =

            new CircularQueue(size);

        for(int i=0;i<size;i++) {

```

```
        System.out.print(
            "Enter element: ");
        q.insert(input.nextInt());
    }

    System.out.println(
        "Removed: " + q.remove());

    System.out.print(
        "Enter New Element: ");
    q.insert(input.nextInt());

    System.out.println(
        "Front Element: "
        + q.peekFront());
}

}
```